학습 교안

하네스 엔지니어링 입문 0단계

AI 에이전트에게 맡기되, 코드 없이 배우는 HTML·CSS 명함 페이지 한 장

OpenCode · OpenRouter · 브라우저

0장 준비 + 1~6장 · 프로그래밍 언어 없이 진행하는 실습

강사 김대철

작성일 2026.11.13

실습 환경 Windows + Git + OpenCode (Python 불필요)

실습 저장소 https://github.com/sinaboro/harness-beginner-tutorial

교안의 명령과 테스트 결과는 Windows PowerShell 에서 실제로 실행해 얻은 것입니다.
AI 의 답변은 실행할 때마다 달라지므로, 교안에는 답변 대신 확인 방법을 실었습니다.

목차

과정 안내 . . . 5

0단계 준비하기 (도커) . . . 6

1. Docker Desktop 설치 (Windows) . . . 6
2. 저장소 받기와 설정 파일 (Windows) . . . 6
3. 작업 환경 만들기 (Windows, 처음 한 번) . . . 6
4. 컨테이너 들어가기 (Windows) . . . 7
5. OpenRouter 연결 (컨테이너 안) . . . 7
6. 작업 폴더 확인 . . . 7

확인 체크리스트 . . . 7

자주 만나는 오류 . . . 8

0단계 1장. AI 와 하네스 체험 . . . 9

이 장에서 할 일 한눈에 보기 . . . 9

1단계. 실험 A — 조건 없이 한 줄로 시키기 (20분) . . . 9

2단계. 실험 A 결과 확인하고 표 채우기 (15분) . . . 9

3단계. 실험 B — 조건 세 개를 붙여서 시키기 (25분) . . . 10

4단계. 표 B 채우고 표 A 와 비교하기 (20분) . . . 10

5단계. 조건 세 개가 막으려는 것 적기 (10분) . . . 11

확인 체크리스트 . . . 11

자주 만나는 오류 . . . 11

0단계 2장. 말로 된 요구사항 쓰기 . . . 12

이 장에서 할 일 한눈에 보기 . . . 12

1단계. 흐린 말 고치기 (15분) . . . 12

2단계. 요구사항 파일 만들기 (20분) . . . 12

3단계. 워크시트대로 칸 채우기 (35분) . . . 13

4단계. 체크리스트로 점검하고 고치기 (15분) . . . 14

5단계. 옆 사람과 바꿔 읽고 질문 적기 (15분) . . . 14

확인 체크리스트 . . . 14

자주 만나는 오류 . . . 14

0단계 3장. 작업 지시서 쓰기 . . . 15

이 장에서 할 일 한눈에 보기 . . . 15

1단계. 지시서 파일 만들기 (5분) . . . 15

2단계. 여섯 칸 워크시트 채우기 (30분) . . . 15

3단계. AI 에게 지시서 건네기 (15분) . . . 16

4단계. 계획 검토하기 (20분) . . . 16

5단계. 결과와 지시서 대조하기 (20분) . . . 17

6단계. 한 줄 지시와 지시서 지시 비교하기 (20분) . . . 17
확인 체크리스트 . . . 17
자주 만나는 오류 . . . 18
0단계 4장. 규칙과 권한 세우기 . . . 19
이 장에서 할 일 한눈에 보기 . . . 19
1단계. 작업 폴더에 파일 세 개 복사하기 (5분) . . . 19
2단계. AGENTS.md 의 [작성] 칸 채우기 (30분) . . . 19
3단계. 권한 설정 읽고 답하기 (15분) . . . 20
4단계. 막혀야 할 일을 AI 에게 시키고 결과 적기 (25분) . . . 21
5단계. 규칙이 지켜지는지 확인하기 (15분) . . . 22
확인 체크리스트 . . . 22
자주 만나는 오류 . . . 22
0단계 5장. 체크리스트로 확인하고 저장하기 . . . 23
이 장에서 할 일 한눈에 보기 . . . 23
1단계. AI 에게 명함 만들게 하기 (25분) . . . 23
2단계. 확인기록 파일 만들기 (5분) . . . 24
3단계. 확인기록의 항목 ①~④ 확인하기 (15분) . . . 24
4단계. 확인기록의 항목 ⑤~⑦ 확인하기 (10분) . . . 24
5단계. 확인기록의 항목 ⑧~⑭ 확인하기 (25분) . . . 25
6단계. 실패한 항목 고치게 하기 (15분) . . . 25
7단계. 마지막 두 항목 확인하고 저장하기 (15분) . . . 25
확인 체크리스트 . . . 26
자주 만나는 오류 . . . 26
0단계 6장. 미니 프로젝트: 자기소개 명함 페이지 . . . 27
이 장에서 할 일 한눈에 보기 . . . 27
1단계. 작업 폴더 만들고 앞 장의 파일 옮기기 (15분) . . . 27
2단계. 요구사항이 내 정보로 채워졌는지 대조하기 (15분) . . . 28
3단계. AI 에게 맡기고 계획 검토하기 (25분) . . . 28
4단계. 확인기록의 16개 항목 확인하기 (30분) . . . 29
5단계. 과정 기록 쓰고 저장하기 (25분) . . . 29
확인 체크리스트 . . . 30
막혔을 때 . . . 31
다음 단계 . . . 31
부록. 템플릿과 예제 . . . 32
A-1. 명함 요구사항 (2장) . . . 32
A-2. 명함 작업 지시서 (3장·6장) . . . 33
A-3. 규칙 문서 빈칸 틀 (4장) . . . 34
A-4. 권한 설정 (4장) . . . 35

A-5. 확인 체크리스트 (5장) 36

과정 안내

0단계는 프로그래밍 언어를 배우지 않고, AI 에이전트에게 맡기면서 하네스의 원리를 익히는 과정입니다. 결과물은 HTML 과 CSS 로 만든 자기소개 명함 페이지 한 장입니다. 1단계(Python 과 자동 테스트)로 넘어가기 전에 이 과정을 먼저 합니다.

0단계 준비하기 (도커)

시간	1시간 30분 (처음 한 번)
준비물	Windows 10·11, Docker Desktop, Git for Windows, VS Code, 인터넷, OpenRouter 계정
결과	도커 컨테이너 안에서 OpenCode 가 OpenRouter 에 연결되고, 작업 폴더가 Windows 와 연결된다

0단계는 프로그래밍 언어를 배우지 않습니다. 실행 환경은 **도커 컨테이너**입니다. PC 에 Python 이나 OpenCode 를 따로 설치하지 않습니다.

> **역할 나누기**
> - **Windows 쪽(PowerShell)**: 설치하고, 저장소를 받고, 컨테이너를 켠다. 명령 블록에 `[Windows]` 라고 표시합니다.
> - **컨테이너 쪽(bash)**: 교안의 실습 명령을 친다. 명령 블록에 `[컨테이너]` 라고 표시합니다.
> - **편집과 확인**: Windows 의 VS Code 에서 파일을 고치고, 브라우저로 결과를 봅니다.

1. Docker Desktop 설치 (Windows)

1. https://www.docker.com/products/docker-desktop/ 에서 받아 설치합니다.
2. 설치 중 **Use WSL 2 instead of Hyper-V** 가 보이면 체크된 채로 둡니다.
3. PC 를 재시작하고 Docker Desktop 을 실행합니다. 왼쪽 아래가 **Engine running** 이 될 때까지 기다립니다.

[Windows] (PowerShell)

```
docker version
```

Client 와 Server 두 줄이 모두 나오면 됩니다. Server 가 없으면 Docker Desktop 이 덜 켜진 것입니다.

> WSL 오류가 나면 관리자 PowerShell 에서 `wsl --update` 를 실행하고 재부팅합니다. BIOS 의 가상화(VT-x 또는 AMD-V)가 꺼져 있으면 켭니다.

2. 저장소 받기와 설정 파일 (Windows)

[Windows] (PowerShell)

```
git clone https://github.com/sinaboro/harness-beginner-tutorial C:\harness-beginner-tutorial
New-Item -ItemType Directory C:\harness-lab
cd C:\harness-beginner-tutorial\docker
Copy-Item .env.example .env
```

VS Code 로 `C:\harness-beginner-tutorial\docker\.env` 를 열어 `GIT_NAME` 과 `GIT_EMAIL` 을 본인 것으로 바꾸고 저장합니다.

3. 작업 환경 만들기 (Windows, 처음 한 번)

[Windows] (PowerShell)

```
cd C:\harness-beginner-tutorial\docker
docker compose build
```

처음에는 몇 분 걸립니다. 마지막에 `Built` 또는 `Successfully` 가 보이면 됩니다.

4. 컨테이너 들어가기 (Windows)

[Windows] (PowerShell)

```
docker compose run --rm workspace
```

프롬프트가 `root@…:/workspace#` 로 바뀌면 컨테이너 안입니다. 이제 아래는 **컨테이너 안(bash)** 에서 칩니다.

[컨테이너]

```
pwd
ls /tutorial/templates/stage0
git --version
python --version
opencode --version
```

`/workspace` 는 Windows 의 `C:\harness-lab` 과 같은 폴더입니다. `/tutorial` 에는 교안 저장소가 보입니다(읽기 전용).

5. OpenRouter 연결 (컨테이너 안)

OpenRouter 키 만들기는 https://openrouter.ai 에서 합니다. 키는 비밀번호와 같으니 채팅이나 화면 공유에 보여주지 않습니다.

[컨테이너]

```
opencode
```

입력 칸에 `/connect` 를 치고 **OpenRouter** 를 고른 뒤 키를 붙여 넣습니다. `/models` 로 목록이 나오면 `/exit` 로 나옵니다.

> 연결 정보는 도커 볼륨에 남습니다. 다음에 `docker compose run --rm workspace` 로 다시 들어가도 키를 다시 넣을 필요가 없습니다.

6. 작업 폴더 확인

[컨테이너]

```
mkdir -p /workspace/card4
ls -la /workspace
```

`/workspace/card4` 가 Windows 의 `C:\harness-lab\card4` 로도 보이면 연결이 된 것입니다. Windows 탐색기에서 확인합니다.

확인 체크리스트

- ■ `docker version` 에 Server 가 나온다 (Windows)
- ■ 컨테이너 안에서 `opencode --version` 이 나온다
- ■ 컨테이너 안에서 `opencode` → `/models` 목록이 나온다
- ■ Windows 의 `C:\harness-lab\card4` 가 보인다

자주 만나는 오류

증상	해결
`error during connect`	Docker Desktop 이 Engine running 이 아니다. 기다리거나 재시작한다.
`/workspace` 가 비어 있다	`.env` 의 `HARNESS_LAB_DIR` 이 틀렸다. `C:/harness-lab` 처럼 슬래시로 쓰고 폴더가 있는지 본다.
`git commit` 이 `Please tell me who you are`	`.env` 의 `GIT_NAME`, `GIT_EMAIL` 을 채우고 컨테이너를 다시 들어간다.
`/models` 목록이 비어 있다	키를 잘못 붙였거나 크레딧이 없다. OpenRouter 의 Keys, Credits 를 확인한다.

자세한 설명은 도커로 실행하기를 봅니다.

0단계 1장. AI 와 하네스 체험

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	1시간 30분
준비물	00 준비하기를 마친 상태 (컨테이너 안에서 `opencode` 가 `/models` 목록을 보여 준다)
결과	조건 없이 시킨 결과와 조건을 붙여 시킨 결과를 표로 비교하고, 차이를 한 문장으로 말한다

이 장에서 할 일 한눈에 보기

1. 실험 A 폴더를 만들고, AI 에게 한 줄로 시킨다. (20분)
2. 실험 A 가 만든 파일을 확인하고 표 A 를 채운다. (15분)
3. 실험 B 폴더를 만들고, 조건 세 개를 붙여 시킨다. (25분)
4. 표 A 와 표 B 를 같은 항목끼리 비교해 차이를 적는다. (20분)
5. 조건 세 개가 막으려는 것을 적는다. (10분)

1단계. 실험 A — 조건 없이 한 줄로 시키기 (20분)

무엇을: 실험용 폴더를 만들고, OpenCode 에 문장 한 줄을 입력한다.

[컨테이너]

```
mkdir -p /workspace/card-a
cd /workspace/card-a
opencode
```

OpenCode 입력 칸에 아래 문장을 **그대로** 붙여 넣고 Enter 를 누릅니다. 다른 말은 덧붙이지 않습니다.

■ [AI] OpenCode 입력 칸

```
내 자기소개 페이지를 HTML 로 만들어 줘.
```

AI 가 작업을 끝낼 때까지 기다립니다. 끝나면 입력 칸에 `/exit` 를 치고 Enter 를 누릅니다.

2단계. 실험 A 결과 확인하고 표 채우기 (15분)

무엇을: AI 가 만든 파일을 확인하고, 아래 표의 빈칸에 **보이는 그대로** 적는다.

[컨테이너]

```
ls -la /workspace/card-a
```

확인 방법:

- 파일 목록에 있는 파일 이름을 표의 2번 칸에 적는다.
- 파일을 VS Code 에서 열어 `<img`, `<script`, `http` 가 들어 있는지 찾는다. 있으면 "있음", 없으면 "없음"을 적는다.
- 브라우저 확인은 Windows 에서 한다. `C:\harness-lab\card-a\index.html` 을 더블클릭해서 열고, 페이지에 **내 이름**이 있는지 본다.

항목	내가 확인해서 적은 내용
1. 작업 전에 계획을 말했는가? (대화 기록에 계획이 있는가)	
2. 만든 파일 이름은? (ls 결과)	
3. 내 이름이 들어 있는가? (있음 / 없음 / 가짜 이름이 들어 있음)	
4. 그림, 외부 파일, 스크립트(<img, <script, http)가 있는가?	
5. 완성했다고 말할 때, 무엇을 보고 확인했는가? (AI 의 말 한 줄을 그대로 적는다)	

3단계. 실험 B — 조건 세 개를 붙여서 시키기 (25분)

무엇을: 실험 B 폴더를 만들고, 아래 문장을 **그대로** 입력한다.

[컨테이너]

```
mkdir -p /workspace/card-b
cd /workspace/card-b
opencode
```

■ [AI] OpenCode 입력 칸

```
자기소개 페이지를 HTML 과 CSS 로 만들어 줘.
조건 1. 파일은 index.html 과 style.css 두 개만 만든다.
조건 2. 그림, 외부 글꼴, 자바스크립트는 쓰지 않는다.
조건 3. 먼저 내 이름과 취미를 물어보고, 답을 받은 뒤에 만든다. 지금은 질문만 해 줘.
```

AI 가 질문하면 아래 답을 **그대로** 입력합니다.

■ [AI] OpenCode 입력 칸

```
이름은 홍길동, 소개는 "배우는 것을 좋아하는 사람", 취미는 독서, 산책, 요리입니다. 이제 만들어 줘.
```

(이름과 소개, 취미는 본인의 것으로 바꿔도 됩니다.) AI 가 끝나면 `/exit` 를 칩니다.

4단계. 표 B 채우고 표 A 와 비교하기 (20분)

무엇을: 실험 B 결과를 확인하고, 표 A 와 같은 항목을 나란히 적는다.

[컨테이너]

```
ls -la /workspace/card-b
```

확인 방법은 2단계와 같습니다. 파일 이름, 내 이름 확인, 외부 파일 확인을 합니다.

항목	실험 A (표 A 의 답)	실험 B (이번 답)	같은가, 다른가
1. 계획을 먼저 말했는가?			
2. 파일 개수와 이름			
3. 내 이름이 들어 있는가?			
4. 그림·외부 파일·스크립트가 있는가?			

비교 방법:

- 같은 항목의 두 칸을 읽고, 다르면 "다름"이라고 적는다.
- 차이가 가장 큰 항목 **한 개**에 동그라미를 친다.

5단계. 조건 세 개가 막으려는 것 적기 (10분)

무엇을: 실험 B 에서 붙인 조건 세 개가 **각각 무엇을 막으려고 한 것인지** 적는다.

조건	막으려는 것 (한 문장)
조건 1 (파일 두 개만)	
조건 2 (그림, 외부 글꼴, 스크립트 금지)	
조건 3 (먼저 묻기)	

확인 기준: 각 칸의 문장에 "AI 가 ~하지 못하게" 또는 "~가 생기지 않게"라는 뜻이 들어 있어야 합니다. 들어 있지 않으면 다시 씁니다.

확인 체크리스트

- ■ 표 A 의 다섯 항목을 모두 채웠다.
- ■ 표 B 의 네 항목을 모두 채웠다.
- ■ 같은 항목끼리 "같다/다르다"를 적었다.
- ■ 조건 세 개가 막으려는 것을 각각 한 문장으로 적었다.
- ■ 차이가 가장 큰 항목 한 개에 동그라미를 쳤다.

자주 만나는 오류

증상	해결
`opencode` 가 "모델을 선택하세요"라고 한다	00 준비하기 5번의 `/connect` 를 다시 한다.
AI 가 질문하지 않고 바로 파일을 만든다 (실험 B)	조건 3 을 한 글자도 바꾸지 않고 다시 입력한다. 그래도 그렇다면 `/new` 를 치고 처음부터 다시 입력한다.
브라우저에서 페이지가 안 열린다	파일 이름이 `index.html` 인지 확인한다. 확장자가 `.txt` 로 바뀌었으면 VS Code 에서 파일 이름을 고친다.

0단계 2장. 말로 된 요구사항 쓰기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	1시간 30분
준비물	1장 완료
결과	내 명함 페이지의 요구사항 파일 `내명함.md` 를 칸 채우기 방식으로 완성한다

이 장에서 할 일 한눈에 보기

1. 흐린 말을 확인할 수 있는 말로 바꾼다. (15분)
2. 요구사항 파일 `내명함.md` 를 만든다. (20분)
3. 칸을 하나씩 채운다. 워크시트를 따른다. (35분)
4. 체크리스트로 점검하고 고친다. (15분)
5. 옆 사람과 바꿔 읽고 질문을 적는다. (15분)

1단계. 흐린 말 고치기 (15분)

무엇을: 아래 문장에서 흐린 낱말을 찾아 표에 옮기고, 각각을 **숫자나 눈으로 볼 수 있는 말**로 바꿔 씁니다.

> 명함은 예쁘고, 보기 좋고, 누구나 쉽게 읽을 수 있어야 한다.

흐린 말	눈으로 확인할 수 있는 말로 바꾼 것 (예: "바탕은 연한 회색이다")
예쁘고	
보기 좋고	
쉽게 읽을 수 있어야	

확인 기준: 바꾼 말을 읽고 "눈으로 보면 '맞다/아니다'를 말할 수 있는가?"에 답합니다. 답할 수 없으면 다시 바꿉니다.

2단계. 요구사항 파일 만들기 (20분)

무엇을: 컨테이너 안에 작업 폴더를 만들고, 빈 파일 `내명함.md` 를 만듭니다. (파일 내용은 3단계에서 씁니다.)

[컨테이너]

```
mkdir -p /workspace/card-req
cd /workspace/card-req
touch 내명함.md
ls -la
```

[Windows] VS Code 에서 `C:\harness-lab\card-req\내명함.md` 를 엽니다.

> 예시 파일은 `/tutorial/templates/stage0/REQ_명함.md` 입니다(Windows 에서는 `C:\harness-beginner-tutorial\templates\stage0\REQ_명함.md`). **보기만 하고 복사하지 않습니다.** 내 파일은 직접 씁니다.

3단계. 워크시트대로 칸 채우기 (35분)

무엇을: 내명함.md 에 아래 틀을 **그대로** 붙여 넣고, [ ] 로 된 칸만 채웁니다. 다른 줄은 바꾸지 않습니다.

```
# 요구사항: 내 명함 페이지

## 기능 요구사항

### R1. 이름과 한 줄 소개
- 이렇게 하면: 페이지를 브라우저로 연다.
- 이런 결과가 나온다: 맨 위에 [내 이름]이 가장 큰 글씨로 보이고, 그 아래 [한 줄 소개]가 보인다.
- 잘못 쓰면: 이름이나 소개가 비어 있으면, 그 줄은 보이지 않고 페이지는 깨지지 않는다.

### R2. 취미 목록
- 이렇게 하면: 페이지를 브라우저로 연다.
- 이런 결과가 나온다: "취미"라는 제목 아래에 취미가 [세 개] 세로로 보인다. 취미 이름은 [취미1], [취미2], [취미3] 이다
  .
- 잘못 쓰면: 취미가 세 개가 아니어도 목록이 깨지지 않는다.

### R3. 연락 링크
- 이렇게 하면: "이메일 보내기" 링크를 누른다.
- 이런 결과가 나온다: 이메일 작성 창이 열리고, 받는 사람은 [내 이메일 주소] 이다.
- 잘못 쓰면: 이메일 주소가 비어 있으면 링크를 보여 주지 않는다.

### R4. 좁은 화면에서도 보기 좋게
- 이렇게 하면: 브라우저 창의 오른쪽 끝을 잡고 왼쪽으로 줄인다.
- 이런 결과가 나온다: 글과 목록이 가로로 잘리지 않고 세로로 쌓인다.
- 잘못 쓰면: 가로 스크롤바가 생기면 실패다.

### R5. 색과 글꼴
- 이렇게 하면: 페이지를 연다.
- 이런 결과가 나온다: 바탕은 [연한 회색 계열], 이름은 [진한 남색 계열]이다. 이름이 가장 큰 글씨다.

## 만들지 않는 것
- 로그인, 방명록, 입력 폼
- 그림 파일과 외부 글꼴
- 자바스크립트
```

칸 채우기 규칙 (대괄호 [ ] 칸마다 하나씩):

칸	무엇을 쓰나	예
[내 이름] (R1)	내 이름 한 개	홍길동
[한 줄 소개] (R1)	짧은 문장 한 개	배우는 것을 좋아하는 사람
[세 개] (R2)	그대로 둔다	세 개
[취미1], [취미2], [취미3] (R2)	취미 한 개씩	독서, 산책, 요리
[내 이메일 주소] (R3)	내 이메일 주소	you@example.com
[연한 회색 계열], [진한 남색 계열] (R5)	이 두 칸은 바꾸지 않는다	(그대로)

하는 법: VS Code 에서 [내 이름] 을 찾아(Ctrl+F) 내 이름으로 바꿉니다. 칸마다 같은 방법으로 바꾸고, 대괄호 [ ] 는 모두 지웁니다.

확인 기준: [또는] 가 남아 있으면 칸을 다 채우지 않은 것입니다. 저장하고 다음 단계로 갑니다.

4단계. 체크리스트로 점검하고 고치기 (15분)

무엇을: 아래 여섯 항목을 **한 줄씩** 확인하고, 아니오가 있으면 `내명함.md` 를 고칩니다.

확인할 항목	확인 방법	예 / 아니오
1. 대괄호 `[ ]` 가 한 개도 남아 있지 않은가?	Ctrl+F 로 `[` 를 검색해 0건인지 본다	
2. R1~R5 각각에 "이렇게 하면", "이런 결과가 나온다", "잘못 쓰면" 세 줄이 있는가?	다섯 번 세어 본다	
3. "이런 결과가 나온다" 줄에 "예쁘다, 잘 보인다, 편하다" 같은 흐린 말이 없는가?	세 낱말을 검색해 0건인지 본다	
4. "만들지 않는 것"이 적혀 있는가?	4번째 줄 근처를 본다	
5. R3 의 이메일 주소에 `@` 가 들어 있는가?	검색해서 있는지 본다	
6. R2 의 취미가 세 개인가?	`[취미` 를 검색해 세 번 나오는지 본다	

아니오가 있으면, 그 항목을 고치고 그 항목의 번호를 옆에 적습니다.

5단계. 옆 사람과 바꿔 읽고 질문 적기 (15분)

무엇을: 옆 사람의 `내명함.md` 를 읽고, 질문을 **두 개 이상** 적습니다.

질문은 아래 형식으로 씁니다.

```
R[번호] 에서 [무엇]이 궁금합니다. 예: "R1 에서 소개가 두 줄이면 어떻게 되나요?"
```

질문에 답이 적혀 있지 않으면, 그 항목은 빈틈입니다. 빈틈이 생긴 줄을 찾아 **그 줄을 한 문장 더 구체적으로** 고칩니다.

확인 체크리스트

- ■ 1단계 표 세 줄을 모두 채웠다.
- ■ `내명함.md` 가 있고, 대괄호 `[ ]` 가 남아 있지 않다.
- ■ 4단계 여섯 항목을 모두 확인하고, 아니오는 고쳤다.
- ■ 옆 사람의 파일에서 질문을 두 개 이상 적었다.

자주 만나는 오류

증상	해결
`touch` 명령이 없다고 나온다	명령을 컨테이너 안에서 쳤는지 확인한다. (Windows 에서 치면 안 된다)
`내명함.md` 가 안 보인다	컨테이너 안에서 `ls -la` 를 친다. 없다면 `cd /workspace/card-req` 를 한 뒤 다시 `touch 내명함.md` 를 친다.
파일을 저장했는데 바뀌지 않은 것 같다	VS Code 에서 Ctrl+S 를 눌렀는지 확인한다. 탭 제목 옆에 점이 있으면 저장되지 않은 것이다.

0단계 3장. 작업 지시서 쓰기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	1시간 30분
준비물	2장의 `내명함.md`
결과	지시서 `지시서.md` 를 여섯 칸 워크시트로 완성하고, AI 가 낸 계획을 검토해 승인한다

이 장에서 할 일 한눈에 보기

1. 지시서 파일을 만든다. (5분)
2. 여섯 칸을 워크시트대로 채운다. (30분)
3. AI 에게 지시서를 건네고, 계획을 받는다. (15분)
4. 계획을 체크리스트로 검토하고, 고치게 하거나 승인한다. (20분)
5. 한 줄 지시와 지시서 지시의 차이를 표로 적는다. (20분)

1단계. 지시서 파일 만들기 (5분)

[컨테이너]

```
mkdir -p /workspace/card-task
cd /workspace/card-task
touch 지시서.md
ls -la
```

[Windows] VS Code 에서 `C:\harness-lab\card-task\지시서.md` 를 엽니다.

2단계. 여섯 칸 워크시트 채우기 (30분)

무엇을: `지시서.md` 에 아래 틀을 **그대로** 붙여 넣고, `[ ]` 칸을 채웁니다. 칸마다 "무엇을 쓰는지"와 "예"가 있으니 그대로 따릅니다.

```
# 작업 지시서: 내 명함 페이지

## 1. 목표
[끝나면 무엇이 되어 있어야 하는지 한 문장]

## 2. 요구사항
[2장에서 쓴 내명함.md 의 요구사항 번호]

## 3. 범위
- 새로 만드는 파일: [파일 이름 두 개]
- 고치면 안 되는 파일: [파일 이름 두 개]

## 4. 하지 말 것
- [하지 말아야 할 것 한 가지]
- [하지 말아야 할 것 한 가지]
- [하지 말아야 할 것 한 가지]

## 5. 완료 조건
- [ ] [확인할 수 있는 조건 한 가지]
```

```
- [ ] [확인할 수 있는 조건 한 가지]

## 6. 진행 방법
먼저 계획만 말한다. 내가 "진행"이라고 하기 전에는 파일을 만들지 않는다.
끝나면 만든 파일 이름과, 완료 조건 중 어느 것을 확인했는지 보고한다.
```

칸별 워크시트:

칸	무엇을 쓰나	예 (그대로 쓰지 말고 내 내용으로 바꾼다)
1. 목표	파일 두 개로 된 명함 페이지 한 개	내명함.md 의 R1~R5 를 만족하는 명함 페이지를 index.html 과 style.css 로 만든다
2. 요구사항	2장 파일의 번호를 그대로	R1, R2, R3, R4, R5
3. 범위 (새로 만드는 파일)	이름 두 개	index.html, style.css
3. 범위 (고치면 안 되는 파일)	이름 두 개	CHECKLIST.md, AGENTS.md
4. 하지 말 것	2장 "만들지 않는 것"에서 세 개를 골라 쓴다	자바스크립트, 그림 파일, 외부 글꼴
5. 완료 조건	눈으로 볼 수 있는 조건 두 개	파일이 두 개뿐이다 / 브라우저에서 이름이 가장 크게 보인다
6. 진행 방법	이미 적혀 있다. 고치지 않는다	(그대로)

확인 기준: 아래 세 가지를 보고 아니오가 있으면 고칩니다.

- [가 남아 있지 않다.
- 5번 완료 조건이 "잘 보인다"처럼 흐리지 않다. 눈으로 보고 맞다/아니다를 말할 수 있다.
- 4번 하지 말 것에 적은 것이 2장 "만들지 않는 것"에 있다.

3단계. AI 에게 지시서 건네기 (15분)

[컨테이너]

```
cd /workspace/card-task
opencode
```

OpenCode 입력 칸에 아래 문장을 **그대로** 붙여 넣습니다.

■ [AI] OpenCode 입력 칸

```
지시서.md 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고, 내가 "진행"이라고 할 때까지 파일을 만들지 마.
```

AI 가 계획을 말하면 4단계로 갑니다. (아직 "진행"이라고 하지 않습니다.)

4단계. 계획 검토하기 (20분)

무엇을: AI 의 계획을 아래 표로 한 줄씩 검토합니다.

확인 질문	답 (예/아니오)	아니오면 할 일
1. 계획이 만드는 파일이 index.html 과 style.css 두 개뿐인가?		
2. 계획에 자바스크립트, 그림, 외부 글꼴이 들어 있지 않은가?		
3. 계획이 CHECKLIST.md 나 AGENTS.md 를 고치겠다고 하지 않는가?		
4. 계획에 2장 요구사항 R1~R5 가 모두 들어 있는가?		

아니오가 있으면, 아래 문장을 그대로 붙여 넣고, 아니오 항목 번호만 바꿉니다.

■ [AI] OpenCode 입력 칸

> 계획 중 [번호]번이 지시서의 "하지 말 것" 또는 "범위"와 맞지 않아. 그 부분을 빼고 계획을 다시 말해 줘. 아직 파일은 만들지 마.

모든 답이 예이면 아래 문장을 입력합니다.

■ [AI] OpenCode 입력 칸

> 진행

끝나면 `/exit` 를 칩니다.

5단계. 결과와 지시서 대조하기 (20분)

[컨테이너]

```
cd /workspace/card-task
ls -la
```

무엇을: 만들어진 파일을 지시서의 3번 칸과 대조합니다.

지시서 항목	실제 결과	같은가?
3번 새로 만드는 파일 두 개		
3번 고치면 안 되는 파일 (CHECKLIST.md 등)		
4번 하지 말 것 (그림, 스크립트 등이 있는가)		

Windows 에서 확인: `C:\harness-lab\card-task\index.html` 을 더블클릭해 브라우저로 엽니다. 5번 완료 조건 첫 항목을 눈으로 확인합니다. 결과를 표에 적습니다.

6단계. 한 줄 지시와 지시서 지시 비교하기 (20분)

무엇을: 1장 실험 A 의 결과와 이번 결과를 같은 항목끼리 적습니다.

항목	1장 실험 A (한 줄 지시)	3장 (지시서)
계획을 먼저 말했는가?		
시키지 않은 것이 있었는가?		
완료를 확인할 방법이 있었는가?		

비교 기준: 같은 항목의 두 칸에 "있었다/없었다" 또는 "예/아니오" 로 적어야 합니다. 설명이 길면 한 단어로 줄입니다.

확인 체크리스트

- ■ `지시서.md` 의 여섯 칸을 모두 채웠다. `[` 가 남지 않았다.
- ■ 계획 검토 네 항목을 모두 답했고, 아니오는 고쳤다.
- ■ 5단계의 세 항목을 대조했다.
- ■ 6단계 비교표를 채웠다.

자주 만나는 오류

증상	해결
AI 가 `지시서.md` 를 찾지 못한다	명령을 `cd /workspace/card-task` 한 뒤에 `opencode` 를 쳤는지 확인한다.
AI 가 "진행"을 기다리지 않고 만든다	입력한 문장에 "먼저 계획만 말하고"가 있었는지 확인한다. 이미 만들었다면 `/undo` 를 친다.
파일이 다른 폴더에 생겼다	`pwd` 로 현재 폴더를 확인하고, 파일을 같은 폴더로 옮긴다.

0단계 4장. 규칙과 권한 세우기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	1시간 30분
준비물	3장 완료
결과	규칙 파일 `AGENTS.md` 의 [작성] 칸 일곱 개를 채우고, 권한 설정이 막는 일을 직접 확인한다

이 장에서 할 일 한눈에 보기

1. 작업 폴더에 파일 세 개를 복사한다. (5분)
2. `AGENTS.md` 의 [작성] 칸 일곱 개를 정해진 문장으로 채운다. (30분)
3. `opencode.jsonc` 의 규칙 세 개를 읽고 답을 적는다. (15분)
4. AI 에게 막혀야 할 일을 시키고, 결과를 표에 적는다. (25분)
5. `AGENTS.md` 가 실제로 지켜지는지 확인한다. (15분)

1단계. 작업 폴더에 파일 세 개 복사하기 (5분)

[컨테이너]

```
mkdir -p /workspace/card4
cd /workspace/card4
cp /tutorial/templates/stage0/opencode.jsonc .
cp /tutorial/templates/stage0/AGENTS_빈칸.md AGENTS.md
cp /tutorial/templates/stage0/CHECKLIST.md .
ls -la
```

`ls -la` 결과에 `AGENTS.md`, `opencode.jsonc`, `CHECKLIST.md` 세 개가 있으면 됩니다.

2단계. AGENTS.md 의 [작성] 칸 채우기 (30분)

무엇을: Windows 의 VS Code 에서 `C:\harness-lab\card4\AGENTS.md` 를 엽니다. 아래 표는 `[작성]` 칸 **17개**의 순서대로입니다. 칸마다 오른쪽 문장을 **글자 하나도 바꾸지 말고** 그대로 씁니다. 칸 옆의 괄호 설명은 지웁니다.

번호	절	찾을 `[작성]` (파일에서 나오는 모습)	그대로 써야 할 문장
①	1. 이 프로젝트는 무엇인가	`- 목표: [작성]`	내 자기소개 명함 페이지를 index.html 과 style.css 두 파일로 만든다.
②	1. 이 프로젝트는 무엇인가	`- 사용자: [작성] (이 페이지를 볼 사람)`	이 페이지를 처음 보는 사람
③	2. 파일 나누기 (표)	`¦ index.html ¦ [작성] ¦`	고쳐도 된다
④	2. 파일 나누기 (표)	`¦ style.css ¦ [작성] ¦`	고쳐도 된다
⑤	2. 파일 나누기 (표)	`¦ CHECKLIST.md, AGENTS.md, opencode.jsonc ¦ [작성] ¦`	고치면 안 된다
⑥	3. 일하는 순서 1번	`1. [작성] (먼저 무엇을 하는가)`	지시서를 읽고 작업 범위를 확인한다.

번호	절	찾을 [작성] (파일에서 나오는 모습)	그대로 써야 할 문장
⑦	3. 일하는 순서 2번	2. [작성] (계획을 말하고 ...)	계획을 먼저 말하고, 사람이 "진행"이라고 할 때까지 파일을 만들거나 고치지 않는다.
⑧	3. 일하는 순서 3번	3. [작성] (만드는 방법)	자바스크립트, 그림, 외부 글꼴을 쓰지 않고 index.html 과 style.css 를 만든다.
⑨	3. 일하는 순서 4번	4. [작성] (끝났는지 확인하는 방법)	CHECKLIST.md 의 항목을 하나씩 확인한다.
⑩	3. 일하는 순서 5번	5. [작성] (보고하는 방법)	바꾼 파일 이름과 확인한 항목을 보고한다.
⑪	4. 하지 말 것 (첫 줄)	- [작성] (커밋과 저장은 누가 하는가)	저장(git commit)은 하지 않는다. 저장은 사람이 한다.
⑫	4. 하지 말 것 (둘째 줄)	- [작성] (체크리스트에 대해 ...)	CHECKLIST.md 는 고치지 않는다.
⑬	4. 하지 말 것 (셋째 줄)	- [작성] (자바스크립트, 외부 파일, 새 파일에 대해)	자바스크립트, 그림, 외부 파일을 쓰지 않는다.
⑭	5. 쓰기 규칙 (첫 줄)	- [작성] (주석은 어떤 언어로, 어디에 달 것인가)	파일의 모든 줄에 한국어 설명을 단다.
⑮	5. 쓰기 규칙 (둘째 줄)	- [작성] (글꼴과 색은 어떻게 정하는가)	글꼴과 색은 요구사항 R5 를 따른다.
⑯	6. 완료 확인 (첫 줄)	- 완료라고 말하기 전에 [작성] (...)	완료라고 말하기 전에 CHECKLIST.md 의 항목을 하나씩 확인한다.
⑰	6. 완료 확인 (둘째 줄)	- 확인이 안 되면 [작성] (무엇을 하는가)	확인이 안 되면 그 항목을 보고하고 멈춘다.

> 표의 17줄이 파일의 [작성] 17개와 같습니다. 6절에 [작성] 이 두 개 있습니다(⑯, ⑰).

채우는 법:

1. VS Code 에서 Ctrl+F 로 [작성] 을 검색합니다. 첫 번째 [작성] 을 표에서 찾아 문장으로 바꿉니다.
2. 바꾼 뒤 다시 Ctrl+F 로 [작성] 을 검색해 다음 칸으로 갑니다.
3. 17개를 모두 바꾸면 [작성] 검색 결과는 0건이어야 합니다.

확인 기준 (체크 여섯 개): 아래를 Ctrl+F 로 찾아 한 개씩 체크합니다.

- ■ 계획을 먼저 말하고, 사람이 "진행"이라고 할 때까지 파일을 만들거나 고치지 않는다. 가 들어 있다.
- ■ CHECKLIST.md 는 고치지 않는다. 가 들어 있다.
- ■ 저장(git commit)은 하지 않는다. 저장은 사람이 한다. 가 들어 있다.
- ■ 완료라고 말하기 전에 CHECKLIST.md 의 항목을 하나씩 확인한다. 가 들어 있다.
- ■ Ctrl+F 로 [작성] 을 검색하면 **0건**이다.
- ■ - 목표: 줄 뒤에 글자가 있다. (비어 있지 않다)

3단계. 권한 설정 읽고 답하기 (15분)

무엇을: Windows 의 VS Code 에서 C:\harness-lab\card4\opencode.jsonc 를 엽니다. 아래 세 규칙을 찾아 읽고, 칸에 답을 적습니다.

번호	찾을 줄 (이 문장을 Ctrl+F 로 찾는다)	답: 이 규칙이 막는 것은 무엇인가?
1	"resource": "git commit *", "effect": "deny"	
2	"resource": "*CHECKLIST.md", "effect": "deny"	

번호	찾을 줄 (이 문장을 Ctrl+F 로 찾는다)	답: 이 규칙이 막는 것은 무엇인가?
3	`"effect": "ask"` 가 붙은 가장 위의 줄 (`"resource": "*"`)	

확인 기준: 답은 "~을 실행(또는 고치기)할 수 없다", "~을 물어본다" 같은 문장이어야 합니다.

- 1번 답 예: 저장 명령을 실행할 수 없다.
- 2번 답 예: CHECKLIST.md 를 고칠 수 없다.
- 3번 답 예: 명령을 실행하기 전에 나에게 묻는다.

4단계. 막혀야 할 일을 AI 에게 시키고 결과 적기 (25분)

[컨테이너]

```
cd /workspace/card4
git init
opencode
```

OpenCode 입력 칸에 아래 세 문장을 **하나씩** 입력합니다. 한 문장을 입력하고 결과를 적은 뒤 다음 문장으로 갑니다.

■ [AI] OpenCode 입력 칸

```
CHECKLIST.md 의 첫 줄에 "확인함" 이라고 적어 줘.
```

■ [AI] OpenCode 입력 칸

```
git commit -m "test" 를 실행해 줘.
```

■ [AI] OpenCode 입력 칸

```
인터넷에서 명함에 쓸 그림을 찾아서 내려받아 줘.
```

무엇을: 각 문장에 대해 AI 가 **무엇을 했는지** 아래 표에 적습니다.

시킨 일	AI 가 한 일 (예: 실행했다 / 거절했다 / 물어봤다)	파일이 바뀌었는가? (예/아니오)
CHECKLIST.md 에 "확인함" 적기		
git commit 실행		
인터넷에서 그림 찾기		

`/exit` 를 친 뒤 확인합니다.

[컨테이너]

```
git status
git log --oneline
```

확인 기준:

- `git log --oneline` 에 커밋 기록이 **없어야** 합니다. ("does not have any commits yet" 이 나오면 정상)
- `git status` 의 `CHECKLIST.md` 가 **수정됨(modified)** 으로 보이면 AI 가 고친 것입니다. 그 경우 표의 첫 행을 "예"로 적고, AI 에게 원래대로 되돌리게 합니다.

5단계. 규칙이 지켜지는지 확인하기 (15분)

무엇을: AI 에게 규칙을 말하지 않고 물어봅니다. 규칙을 제대로 읽었는지 확인하는 방법입니다.

■ [AI] OpenCode 입력 칸

> 이 폴더에서 작업할 때 지켜야 할 규칙을 파일을 고치지 말고 말로만 알려 줘.

확인 방법: AI 의 답에 아래 세 가지가 있는지 찾습니다.

- ■ "계획을 먼저 말한다" 또는 "진행을 기다린다"
- ■ "CHECKLIST.md 를 고치지 않는다"
- ■ "저장하지 않는다"

셋 중 하나라도 없으면, AI 가 `AGENTS.md` 를 읽지 못한 것입니다. `/exit` 후 `opencode` 를 다시 열고 같은 질문을 합니다. 그래도 없으면 2단계 체크 다섯 항목을 다시 봅니다.

확인 체크리스트

- ■ `AGENTS.md` 에 `[작성]` 이 한 개도 없다.
- ■ 2단계의 다섯 문장이 글자 그대로 들어 있다.
- ■ 3단계 세 답을 적었다.
- ■ 4단계 표를 채웠고, `CHECKLIST.md` 가 바뀌었는지 `git status` 로 확인했다.
- ■ 5단계 세 항목을 확인했다.

자주 만나는 오류

증상	해결
`git init` 이 "Reinitialized" 라고 나온다	이미 저장소라는 뜻이다. 무시해도 된다.
AI 가 `AGENTS.md` 를 언급하지 않는다	파일 위치를 확인한다. `ls -la` 로 `AGENTS.md` 가 `card4` 폴더 안에 있어야 한다.
`git commit` 을 실행하고 커밋이 생겨 버렸다	권한 설정이 막지 못했다는 뜻이다. `git log` 로 확인하고, 강사에게 알린다. 실습 파일은 다시 만들어도 된다.

0단계 5장. 체크리스트로 확인하고 저장하기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	1시간 30분
준비물	4장의 `/workspace/card4` 폴더
결과	AI 가 만든 파일을 체크리스트 항목마다 직접 확인하고, 확인 기록을 남긴 뒤에만 저장한다

이 장에서 할 일 한눈에 보기

1. AI 에게 지시서를 건네고 파일을 만들게 한다. (25분)
2. 체크리스트를 `확인기록.md` 로 복사한다. (5분)
3. 항목 ①~④ 를 확인하고 표에 기록한다. (15분)
4. 항목 ⑤~⑦ 을 확인하고 기록한다. (10분)
5. 항목 ⑧~⑭ 를 확인하고 기록한다. (25분)
6. 실패한 항목을 AI 에게 번호로 알려 고치게 한다. (15분)
7. 항목 ⑮~⑯ 을 확인하고 저장한다. (15분)

중요한 규칙: `CHECKLIST.md` 원본에는 체크 표시를 하지 않습니다. 확인 기록은 **`확인기록.md`** 에만 씁니다.

1단계. AI 에게 명함 만들게 하기 (25분)

[컨테이너]

```
cd /workspace/card4
cp /tutorial/templates/stage0/TASK_명함.md .
ls -la
```

`TASK_명함.md` 가 보이면 됩니다. OpenCode 를 엽니다.

[컨테이너]

```
opencode
```

OpenCode 입력 칸에 아래 문장을 **그대로** 입력합니다.

■ [AI] OpenCode 입력 칸

```
TASK_명함.md 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고, 내가 "진행"이라고 할 때까지 파일을 만들지 마.
```

AI 가 계획을 말하면 4장 3단계의 검토 방식으로 확인합니다.

- 계획이 `index.html`, `style.css` 두 개만 만든다고 하는가? (예/아니오)
- 계획에 CHECKLIST.md 를 고친다는 말이 없는가? (예/아니오)

모두 예이면 아래를 입력합니다. 아니오이면 아니오 내용을 말한 뒤 다시 계획을 받습니다.

■ [AI] OpenCode 입력 칸

> 진행

AI 가 끝나면 `/exit` 를 칩니다.

2단계. 확인기록 파일 만들기 (5분)

[컨테이너]

```
cd /workspace/card4
cp CHECKLIST.md 확인기록.md
ls -la
```

이제 `확인기록.md` 에 체크 표시를 합니다. 원본 `CHECKLIST.md` 는 열어 보기만 합니다.

> **번호 세는 법:** `확인기록.md` 에는 번호가 없습니다. `- [ ]` 로 시작하는 줄을 **위에서부터** 셉니다. 첫 번째 줄이 ①, 두 번째가 ②, 열여섯 번째가 ⑯입니다. 제목 줄(`## …`)은 세지 않습니다.

[Windows] VS Code 에서 `C:\harness-lab\card4\확인기록.md` 를 엽니다.

3단계. 확인기록의 항목 ①~④ 확인하기 (15분)

무엇을: `확인기록.md` 에서 **1단계와 2단계** 항목 네 개(①~④)를 확인합니다. 항목 번호는 `확인기록.md` 에서 위에서부터 붙인 순서입니다.

[컨테이너]

```
cd /workspace/card4
ls -la
```

[Windows] `C:\harness-lab\card4\index.html` 을 **더블클릭**해서 브라우저로 엽니다.

번호	확인기록.md 의 문장 (그대로)	어떻게 확인하나	맞으면 `[x]`
①	폴더에 `index.html` 과 `style.css` 두 파일만 있다.	`ls -la` 결과에 파일이 두 개인지 본다	
②	두 파일을 메모장으로 열어도 글자가 깨져 보이지 않는다.	VS Code 에서 두 파일을 열어 한글을 본다	
③	`index.html` 을 더블클릭했을 때 브라우저에서 열린다.	브라우저 창이 열리는지 본다	
④	페이지 맨 위에 이름이 가장 크게 보인다. (R1)	눈으로 본다. 이름 글씨가 가장 큰가	

`[x]` 는 `확인기록.md` 에서 `- [ ]` 를 `- [x]` 로 바꿔서 표시합니다. **맞을 때만 바꿉니다.**

4단계. 확인기록의 항목 ⑤~⑦ 확인하기 (10분)

무엇을: `확인기록.md` 에서 **2단계 나머지와 3단계** 항목 세 개(⑤~⑦)를 확인합니다.

번호	확인기록.md 의 문장 (그대로)	어떻게 확인하나	맞으면 `[x]`
⑤	이름 아래에 한 줄 소개가 보인다. (R1)	눈으로 본다	
⑥	"취미" 제목 아래에 취미가 세 개 세로로 나열된다. (R2)	브라우저에서 취미 줄을 센다 (세 줄이어야 한다)	
⑦	"이메일 보내기" 링크를 누르면 이메일 작성 창이 열린다. (R3)	링크를 한 번 누른다	

5단계. 확인기록의 항목 ⑧~⑭ 확인하기 (25분)

무엇을: 나머지 항목을 확인합니다.

번호	확인기록.md 의 문장 (그대로)	어떻게 확인하나	맞으면 [x]
⑧	바탕은 연한 색, 이름은 진한 남색이다. (R5)	눈으로 본다	
⑨	브라우저 창을 가장 좁게 줄여도 글이 가로로 잘리지 않는다. (R4)	창의 오른쪽 끝을 잡고 가장 좁게 줄이며 본다	
⑩	가로 스크롤바가 생기지 않는다. (R4)	창 아래에 가로 막대가 보이는지 본다	
⑪	글과 취미 목록이 세로로 쌓여 보인다. (R4)	좁힌 창에서 목록이 한 줄씩 아래로 내려가는지 본다	
⑫	`index.html` 과 `style.css` 에 한국어 주석이 있다.	VS Code 에서 열어 `<!--` 나 `/*` 로 시작하는 한국어 설명을 찾는다	
⑬	자바스크립트(`<script>`)가 없다.	VS Code 에서 `<script` 를 검색한다. 0건이어야 한다	
⑭	외부 파일을 가져오는 줄(`http`, `https`)이 없다.	VS Code 에서 `http` 를 검색한다. 0건이어야 한다	

확인 방법 주의: 창을 줄였는데 글이 잘리면 ⑨ 는 `[ ]` 로 두고 6단계로 갑니다. 보이지 않는 것을 `[x]` 하면 안 됩니다.

6단계. 실패한 항목 고치게 하기 (15분)

무엇을: `[ ]` 로 남은 항목이 있으면, 그 **번호**를 AI 에게 알려 줍니다.

■ [AI] OpenCode 입력 칸

```
확인기록.md 의 [번호]번 항목이 안 돼. 지금 보이는 모습은 [무엇이 보이는지 한 문장]이야. 원인을 먼저 설명하고, 내가 "진행"이라고 하면 ind
  ex.html 과 style.css 만 고쳐 줘. CHECKLIST.md 는 고치지 마.
```

예: `확인기록.md 의 ⑨번 항목이 안 돼. 지금 보이는 모습은 창을 줄이면 이메일 링크가 오른쪽으로 밀려 나가는 것이야.` (번호는 ①처럼 쓰지 말고 `9번`처럼 씁니다.)

AI 가 원인을 설명하면 `진행` 을 입력합니다. 고친 뒤에는 **3단계부터 5단계까지 다시** 확인하고 기록을 갱신합니다.

7단계. 마지막 두 항목 확인하고 저장하기 (15분)

무엇을: `확인기록.md` 의 마지막 두 항목을 확인합니다.

번호	확인기록.md 의 문장 (그대로)	맞으면 [x]
⑮	위의 모든 항목을 확인했다.	1~14번이 모두 `[x]` 일 때만
⑯	바꾼 파일과 확인 방법을 내가 말로 설명할 수 있다.	파일 이름과 확인 방법을 소리 내어 한 번 말해 본 뒤에만

[컨테이너]

```
cd /workspace/card4
grep -c "\[ \]" 확인기록.md
```

`0` 이 나와야 저장합니다. `0` 이 아니면 6단계로 돌아갑니다.

`0` 이면 저장합니다. 저장은 사람이 합니다.

[컨테이너]

```
git add -A
```

```
git status
git commit -m "5장: 명함 페이지 완성 (확인기록 16개 항목 확인)"
git log --oneline
```

`git status` 에 `확인기록.md` 가 보여도 괜찮습니다. 함께 저장합니다.

확인 체크리스트

- `확인기록.md` 에 `[ ]` 가 0건이다.
- `CHECKLIST.md` 원본에는 `[x]` 가 한 개도 없다. (`grep -c "\[x\]" CHECKLIST.md` 가 0)
- 실패 항목이 있었다면, AI 에게 번호로 알리고 고치게 했다.
- `git log --oneline` 에 커밋이 한 줄 생겼다.

자주 만나는 오류

증상	해결
`CHECKLIST.md` 원본에 `[x]` 가 생겼다	원본을 실수로 고친 것이다. `cp /tutorial/templates/stage0/CHECKLIST.md .` 를 쳐서 원본을 다시 복사한다.
`git commit` 이 `Please tell me who you are`	`.env` 의 `GIT_NAME`, `GIT_EMAIL` 을 채우고 컨테이너를 다시 들어간다.
브라우저에서 index.html 이 안 열린다	파일 이름이 `index.html` 인지 확인한다. 탐색기에서 확장자가 보이도록 설정돼 있는지 본다.

0단계 6장. 미니 프로젝트: 자기소개 명함 페이지

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	1~5장 완료. 2장의 `내명함.md` 와 4장의 `AGENTS.md` 가 있다
결과	내 명함 페이지를 AI 에게 맡기고, 확인기록의 16개 항목을 모두 확인한 뒤 저장한다

이 장에서 할 일 한눈에 보기

1. 작업 폴더를 만들고, 앞 장의 파일을 옮겨 온다. (15분)
2. 요구사항 파일이 내 정보로 채워졌는지 대조한다. (15분)
3. AI 에게 맡기고 계획을 검토한다. (25분)
4. 확인기록의 16개 항목을 확인한다. (30분)
5. 과정 기록 `기록.md` 를 채우고 저장한다. (25분)

> 이 장은 **정답 파일을 주지 않습니다.** 요구사항과 체크리스트를 기준으로 확인합니다.

1단계. 작업 폴더 만들고 앞 장의 파일 옮기기 (15분)

[컨테이너]

```
mkdir -p /workspace/my-card
cd /workspace/my-card
cp /workspace/card-req/내명함.md REQUIREMENTS.md
cp /workspace/card4/AGENTS.md .
cp /workspace/card4/opencode.jsonc .
cp /tutorial/templates/stage0/TASK_명함.md .
cp /tutorial/templates/stage0/CHECKLIST.md .
git init
ls -la
```

`ls -la` 결과에 아래 다섯 개가 모두 있어야 합니다.

파일	어디서 왔나	있는가? (예/아니오)
REQUIREMENTS.md	2장의 내명함.md 를 옮긴 것	
AGENTS.md	4장의 AGENTS.md	
opencode.jsonc	4장의 권한 설정	
TASK_명함.md	템플릿 (지시서)	
CHECKLIST.md	템플릿 (확인 기준)	

아니오가 있으면, 그 파일이 있는 폴더를 `ls -la` 로 확인하고 다시 `cp` 합니다.

2단계. 요구사항이 내 정보로 채워졌는지 대조하기 (15분)

무엇을: `REQUIREMENTS.md` 에 대괄호 `[ ]` 가 남아 있지 않은지, 그리고 내 정보가 들어 있는지 확인합니다.

[컨테이너]

```
cd /workspace/my-card
grep -c "\[" REQUIREMENTS.md
grep -n "이메일\|이름\|취미" REQUIREMENTS.md
```

확인 기준:

- 첫 명령의 결과가 `0` 이어야 합니다. `0` 이 아니면 2장 3단계의 칸을 다시 채웁니다.
- 둘째 명령의 결과에 내 이름, 내 취미 세 개, 내 이메일 주소가 보여야 합니다.

대조 방법: `REQUIREMENTS.md` 를 VS Code 로 열고 아래 표를 채웁니다.

항목	파일에 적힌 내용 (그대로 옮겨 적는다)	내 정보와 같은가?
R1 이름		
R1 한 줄 소개		
R2 취미 세 개		
R3 이메일 주소		

`같은가?` 칸이 "아니오"이면 `REQUIREMENTS.md` 의 그 줄을 VS Code 에서 고칩니다. 바꾼 뒤 `grep -c "\[" REQUIREMENTS.md` 를 다시 쳐서 `0` 을 확인합니다.

3단계. AI 에게 맡기고 계획 검토하기 (25분)

[컨테이너]

```
cd /workspace/my-card
opencode
```

OpenCode 입력 칸에 아래 문장을 **그대로** 입력합니다.

■ [AI] OpenCode 입력 칸

```
TASK_명함.md 를 읽고 그 지시대로 작업해 줘. REQUIREMENTS.md 의 내용을 기준으로 삼는다. AGENTS.md 의 작업 순서를 지켜서, 먼저 계획만 말하고 내 승인을 기다려.
```

AI 의 계획을 아래 네 질문으로 검토합니다.

질문	답 (예/아니오)	아니오면 할 일
1. 만드는 파일이 index.html 과 style.css 두 개뿐인가?		
2. 계획에 자바스크립트, 그림, 외부 글꼴이 없는가?		
3. 계획이 CHECKLIST.md, AGENTS.md, REQUIREMENTS.md 를 고친다고 하지 않는가?		
4. 계획이 R1~R5 를 모두 다루는가?		

아니오가 있으면 아래 문장을 입력하고 번호만 바꿉니다.

■ [AI] OpenCode 입력 칸

```
계획 [번호]번이 지시서의 "하지 말 것" 또는 "범위"와 맞지 않아. 그 부분을 빼고 계획을 다시 말해 줘. 아직 파일은 만들지 마.
```

모두 예이면 입력합니다.

■ [AI] OpenCode 입력 칸

```
진행
```

끝나면 `/exit` 를 칩니다.

4단계. 확인기록의 16개 항목 확인하기 (30분)

무엇을: 5장과 **같은 방법**으로 확인합니다. 원본 `CHECKLIST.md` 는 고치지 않고, 복사본 `확인기록.md` 에 체크합니다.

[컨테이너]

```
cd /workspace/my-card
cp CHECKLIST.md 확인기록.md
ls -la
```

[Windows] VS Code 에서 `C:\harness-lab\my-card\확인기록.md` 를 엽니다. 항목은 위에서부터 셉니다. 첫 번째 `- [ ]` 줄이 ①, 열여섯 번째가 ⑯ 입니다.

확인 순서:

1. ①~④: 파일 두 개, 메모장 글자, 더블클릭 열림, 이름이 가장 큼. → 5장 3단계의 방법대로 확인합니다.
2. ⑤~⑦: 한 줄 소개, 취미 세 줄, 이메일 링크. → 취미 줄 수는 브라우저에서 센다. 링크는 한 번 누른다.
3. ⑧~⑭: 색, 창 좁히기, 스크롤바, 세로 쌓임, 한국어 주석, `<script` 없음, `http` 없음. → 5장 5단계의 방법대로 확인합니다.
4. 실패한 번호가 있으면 AI 에게 번호로 알리고 고치게 합니다. 고친 뒤에는 1~3번 순서를 다시 확인합니다.

AI 에게 알리는 문장:

■ [AI] OpenCode 입력 칸

```
확인기록.md 의 [번호]번 항목이 안 돼. 지금 보이는 모습은 [무엇이 보이는지 한 문장]이야. 원인을 먼저 설명하고, 내가 "진행"이라고 하면 ind
  ex.html 과 style.css 만 고쳐 줘. CHECKLIST.md 와 REQUIREMENTS.md 는 고치지 마.
```

⑮와 ⑯은 ⑭까지 모두 `[x]` 인 뒤에만 체크합니다.

[컨테이너]

```
cd /workspace/my-card
grep -c "\[ \]" 확인기록.md
diff CHECKLIST.md /tutorial/templates/stage0/CHECKLIST.md && echo "원본 체크리스트는 그대로입니다"
```

확인 기준: 첫 명령은 `0`, 둘째 명령은 "원본 체크리스트는 그대로입니다"가 나와야 합니다.

5단계. 과정 기록 쓰고 저장하기 (25분)

무엇을: `기록.md` 를 만들고 아래 틀의 빈칸을 채웁니다. 빈칸마다 질문이 있으니, 그 질문에 한 문장으로 답합니다.

[컨테이너]

```
cd /workspace/my-card
touch 기록.md
```

[Windows] VS Code 에서 `C:\harness-lab\my-card\기록.md` 를 열고 아래 틀을 붙여 넣습니다.

```
# 6장 기록

## 1. AI 에게 시킨 일
질문: TASK_명함.md 의 목표 칸을 한 문장으로 옮겨 쓰시오.
답:

## 2. AI 계획에서 고치게 한 것
질문: 계획 검토 네 질문 중 아니오였던 것이 있었나? 있었다면 번호와 무엇을 고치게 했는지 쓰시오. 없었다면 "없음"이라고 쓰시오.
```

```
답:

## 3. 실패한 확인 항목과 고친 방법
질문: 확인기록에서 처음에 체크하지 못한 번호는? 어떻게 고쳤는지 한 문장으로 쓰시오. 없었다면 "없음".
답:

## 4. 내가 이해한 것 세 문장
질문: index.html 과 style.css 가 각각 무엇을 맡는지, 그리고 확인기록이 저장 전에 왜 필요한지 쓰시오.
답:
1.
2.
3.

## 5. 1장 실험 A 와 이번 장의 차이 한 가지
질문: 조건 없이 시켰을 때와 지시서로 시켰을 때, 가장 크게 달라진 것은 무엇인가?
답:
```

확인 기준: 모든 `답:` 칸의 `답:` 뒤에 글자가 있어야 합니다. 질문을 다시 읽고, 빈칸은 한 문장으로 채웁니다.

[컨테이너]

```
cd /workspace/my-card
grep -c "^답:\s*$" 기록.md
```

`grep` 결과가 `0` 이면 모든 답을 채운 것입니다. 결과가 0이 아니면 `답:` 줄 뒤에 아무 글자도 없는 곳이 있습니다. 그 칸을 찾아 채웁니다.

저장합니다. 저장은 사람이 합니다.

[컨테이너]

```
cd /workspace/my-card
git add -A
git status
git commit -m "6장: 나의 자기소개 명함 완성 (확인기록 16개 항목 확인, 기록 작성)"
git log --oneline
```

`git status` 에 `books.json` 같은 실습 파일 말고 `index.html`, `style.css`, `확인기록.md`, `기록.md` 가 보이면 정상입니다.

확인 체크리스트

- ■ 1단계 파일 다섯 개가 모두 있다.
- ■ `REQUIREMENTS.md` 에 `[` 가 0건이고, 내 정보가 들어 있다.
- ■ 계획 검토 네 질문에 모두 답했다.
- ■ 확인기록의 ①~⑯을 모두 `[x]` 했다. (`grep -c "\[ \]"` 가 0)
- ■ `CHECKLIST.md` 원본이 바뀌지 않았다. (`diff` 결과가 없다)
- ■ `기록.md` 의 다섯 답을 모두 썼다.
- ■ `git log --oneline` 에 커밋이 한 줄 생겼다.

막혔을 때

증상	해결
1단계에서 `cp` 가 "No such file" 이라고 한다	앞 장의 폴더 이름을 확인한다. 2장은 `/workspace/card-req`, 4장은 `/workspace/card4` 이다.
AI 가 `REQUIREMENTS.md` 를 고치려 한다	기준은 사람이 고친다. AI 에게 "REQUIREMENTS.md 는 고치지 마"라고 말하고, 그래도 바뀌었다면 `diff` 로 확인해 원래대로 되돌린다.
확인기록의 번호를 헷갈린다	5장의 "번호 세는 법"을 다시 읽는다. 제목 줄은 세지 않는다.
`git commit` 이 `Please tell me who you are`	`.env` 의 `GIT_NAME`, `GIT_EMAIL` 을 채우고 컨테이너를 다시 들어간다.

다음 단계

이것으로 0단계가 끝납니다. 1단계에서는 같은 흐름을 **Python 과 자동 테스트**로 옮겨 갑니다.

부록. 템플릿과 예제

수업에서 복사해 쓰는 파일을 그대로 실었다. 주석을 따라 읽는다.

A-1. 명함 요구사항 (2장)

```
# 요구사항: 자기소개 명함 페이지 (0단계 6장)

## 한 문장 요약
내 이름, 한 줄 소개, 취미, 연락 링크를 보여 주는 한 페이지짜리 웹 명함. 파일 두 개(index.html, style.css)로 만든다.

## 기능 요구사항

### R1. 이름과 한 줄 소개
- 이렇게 하면: 페이지를 브라우저로 연다.
- 이런 결과가 나온다: 맨 위에 이름이 크게 보이고, 그 아래에 한 줄 소개가 보인다.
- 잘못 쓰면: 이름이나 소개 칸을 비워도 페이지가 깨지지 않는다. 비어 있으면 그 줄을 숨긴다.

### R2. 취미 목록
- 이렇게 하면: 페이지를 연다.
- 이런 결과가 나온다: "취미"라는 제목 아래에 취미가 세 개 세로로 나열된다.
- 잘못 쓰면: 취미가 세 개가 아니어도 목록이 깨지지 않는다.

### R3. 연락 링크
- 이렇게 하면: "이메일 보내기" 링크를 누른다.
- 이런 결과가 나온다: 이메일 작성 창이 열린다.
- 잘못 쓰면: 링크 주소가 비어 있으면 링크를 보여 주지 않는다.

### R4. 좁은 화면에서도 보기 좋게
- 이렇게 하면: 브라우저 창을 휴대폰 폭처럼 좁힌다.
- 이런 결과가 나온다: 글과 목록이 가로로 잘리지 않고 세로로 쌓인다.
- 잘못 쓰면: 가로 스크롤바가 생기지 않는다.

### R5. 색과 글꼴
- 이렇게 하면: 페이지를 연다.
- 이런 결과가 나온다: 바탕은 연한 회색, 이름은 진한 남색이다. 글꼴 크기는 이름이 가장 크다.

## 만들지 않는 것
- 로그인, 방명록, 입력 폼
- 그림 파일 (글과 색만 쓴다)
- 자바스크립트 (HTML 과 CSS 만 쓴다)
```

A-2. 명함 작업 지시서 (3장·6장)

```
# 작업 지시서: 자기소개 명함 페이지 (0단계 6장)

## 1. 목표
`REQ_명함.md` 의 R1~R5 를 만족하는 명함 페이지를 HTML 파일 하나와 CSS 파일 하나로 만든다.

## 2. 요구사항
`REQUIREMENTS.md` (이 폴더에 복사해 둔 명함 요구사항)의 R1~R5.

## 3. 범위
- 새로 만드는 파일: `index.html`, `style.css` 두 개뿐이다.
- 고치면 안 되는 파일: `CHECKLIST.md`, `AGENTS.md`, `opencode.jsonc`

## 4. 하지 말 것
- 자바스크립트를 쓰지 않는다.
- 그림 파일이나 외부 글꼴, 외부 CSS 프레임워크(부트스트랩 등)를 가져오지 않는다.
- 요구사항에 없는 섹션(방명록, 입력 폼, 사진)을 덧붙이지 않는다.
- 파일 두 개 외에 다른 파일을 만들지 않는다.

## 5. 완료 조건
- [ ] `index.html` 과 `style.css` 두 파일이 있다.
- [ ] `CHECKLIST.md` 의 항목을 모두 확인했다. (사람이 확인한다)
- [ ] 두 파일 안에 한국어 주석이 있다. (무엇을 꾸미는지 한 줄씩)

## 6. 진행 방법
`AGENTS.md` 의 작업 순서를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 만든다.
끝나면 무엇을 만들었고, `CHECKLIST.md` 의 어느 항목을 확인했는지 보고한다.
```

A-3. 규칙 문서 빈칸 틀 (4장)

```
# AGENTS.md — 작업 규칙 (빈칸을 채우는 틀)

> 4장에서 채웁니다. AI 에이전트(OpenCode)는 이 프로젝트에서 일을 시작할 때 이 파일을 먼저 읽습니다.
> 규칙은 "부탁"입니다. 옆의 권한 설정(`opencode.jsonc`)과 함께 있어야 힘이 생깁니다.
> `[작성]` 을 채우고, 아래 "꼭 들어가야 할 것"을 모두 체크합니다.

## 1. 이 프로젝트는 무엇인가
- 목표: [작성]
- 사용자: [작성] (이 페이지를 볼 사람)

## 2. 파일 나누기
| 파일 | 고쳐도 되는가 |
|---|---|
| index.html | [작성] |
| style.css | [작성] |
| CHECKLIST.md, AGENTS.md, opencode.jsonc | [작성] |

## 3. 일하는 순서
1. [작성] (먼저 무엇을 하는가)
2. [작성] (계획을 말하고 무엇을 기다리는가)
3. [작성] (만드는 방법)
4. [작성] (끝났는지 확인하는 방법)
5. [작성] (보고하는 방법)

## 4. 하지 말 것
- [작성] (커밋과 저장은 누가 하는가)
- [작성] (체크리스트에 대해 무엇을 금지하는가)
- [작성] (자바스크립트, 외부 파일, 새 파일에 대해)

## 5. 쓰기 규칙
- [작성] (주석은 어떤 언어로, 어디에 달 것인가)
- [작성] (글꼴과 색은 어떻게 정하는가)

## 6. 완료 확인
- 완료라고 말하기 전에 [작성] (무엇을 확인하는가. 체크리스트의 어느 부분인가)
- 확인이 안 되면 [작성] (무엇을 하는가)

## 7. 보고 형식
```

[한 일] [바뀐 파일] [확인한 체크리스트 항목] [확인 방법] [남은 일]

```
---

## 꼭 들어가야 할 것 (체크리스트)
- [ ] 계획을 먼저 말하고, 사람의 "진행"을 기다린다는 문장
- [ ] CHECKLIST.md 를 고치지 않는다는 문장
- [ ] 저장(commit)은 하지 않는다는 문장
- [ ] 완료라고 말하기 전 체크리스트를 확인한다는 문장
- [ ] 바뀐 파일과 확인한 항목을 보고한다는 문장
```

A-4. 권한 설정 (4장)

```
// ============================================================
// opencode.jsonc - 0단계 권한 설정 (HTML·CSS 프로젝트용)
//
// 4장에서 프로젝트 맨 위 폴더로 복사합니다.
// 규칙(AGENTS.md)은 "부탁", 이 파일은 "잠금장치"입니다.
//
// 읽는 법
//   action   : 무엇을 하려는가 (shell = 명령 실행, edit = 파일 고치기, read = 파일 읽기)
//   resource : 무엇에 대해서인가 (* 는 "아무 글자나")
//   effect   : allow(그냥 허용) / ask(나에게 묻기) / deny(금지)
//
// 위에서 아래로 읽으며, 여러 규칙에 걸리면 가장 아래 규칙이 이깁니다.
// ============================================================
{
  "$schema": "https://opencode.ai/config.json",
  "model": "openrouter/anthropic/claude-haiku-5.5",
  "share": "disabled",

  "permissions": [
    // ---------- 1. 명령 실행 ----------
    // 기본: 모든 명령은 실행 전에 나에게 묻는다.
    { "action": "shell", "resource": "*", "effect": "ask" },
    // 파일 목록과 Git 상태 확인은 묻지 않는다.
    { "action": "shell", "resource": "dir *", "effect": "allow" },
    { "action": "shell", "resource": "ls *", "effect": "allow" },
    { "action": "shell", "resource": "git status *", "effect": "allow" },
    { "action": "shell", "resource": "git diff *", "effect": "allow" },
    // 저장과 올리기는 사람만 한다.
    { "action": "shell", "resource": "git commit *", "effect": "deny" },
    { "action": "shell", "resource": "git push *", "effect": "deny" },
    { "action": "shell", "resource": "git reset *", "effect": "deny" },

    // ---------- 2. 파일 고치기 ----------
    // 규칙, 권한, 체크리스트(기준)는 AI 가 고치지 못한다.
    { "action": "edit", "resource": "*AGENTS.md", "effect": "deny" },
    { "action": "edit", "resource": "*opencode.jsonc", "effect": "deny" },
    { "action": "edit", "resource": "*CHECKLIST.md", "effect": "deny" },
    { "action": "edit", "resource": "*REQUIREMENTS.md", "effect": "deny" },

    // ---------- 3. 인터넷과 비밀 값 ----------
    { "action": "webfetch", "resource": "*", "effect": "deny" },
    { "action": "websearch", "resource": "*", "effect": "deny" },
    { "action": "read", "resource": "*.env", "effect": "deny" }
  ]
}
```

A-5. 확인 체크리스트 (5장)

```
# 검증 체크리스트 (명함 페이지)

> 이 파일이 "맞는지"를 판단하는 기준입니다. AI 는 이 파일을 고칠 수 없습니다.
> 완료라고 말하기 전에, 사람이 아래를 하나씩 직접 보고 체크합니다.
> 하나라도 빈칸이면 저장하지 않습니다.

## 1단계. 파일 확인 (1분)
- [ ] 폴더에 `index.html` 과 `style.css` 두 파일만 있다. (다른 파일이 생겼다면 AI 에게 되돌리게 한다)
- [ ] 두 파일을 메모장으로 열어도 글자가 깨져 보이지 않는다.

## 2단계. 브라우저로 열어 보기 (3분)
- [ ] `index.html` 을 더블클릭했을 때 브라우저에서 열린다.
- [ ] 페이지 맨 위에 이름이 가장 크게 보인다. (R1)
- [ ] 이름 아래에 한 줄 소개가 보인다. (R1)

## 3단계. 내용 확인 (3분)
- [ ] "취미" 제목 아래에 취미가 세 개 세로로 나열된다. (R2)
- [ ] "이메일 보내기" 링크를 누르면 이메일 작성 창이 열린다. (R3)
- [ ] 바탕은 연한 색, 이름은 진한 남색이다. (R5)

## 4단계. 좁은 화면 확인 (2분)
- [ ] 브라우저 창을 가장 좁게 줄여도 글이 가로로 잘리지 않는다. (R4)
- [ ] 가로 스크롤바가 생기지 않는다. (R4)
- [ ] 글과 취미 목록이 세로로 쌓여 보인다. (R4)

## 5단계. 코드 읽기 (5분)
- [ ] `index.html` 과 `style.css` 에 한국어 주석이 있다. (무엇을 꾸미는지 한 줄씩)
- [ ] 자바스크립트(`<script>`)가 없다.
- [ ] 외부 파일을 가져오는 줄(`http`, `https`)이 없다.

## 6단계. 저장 전 마지막 확인 (1분)
- [ ] 위의 모든 항목을 확인했다.
- [ ] 바꾼 파일과 확인 방법을 내가 말로 설명할 수 있다.

---

**체크를 다 했으면** 이 파일의 항목을 모두 체크한 상태로 저장합니다. 저장은 사람이 합니다. AI 는 저장하지 않습니다.
```